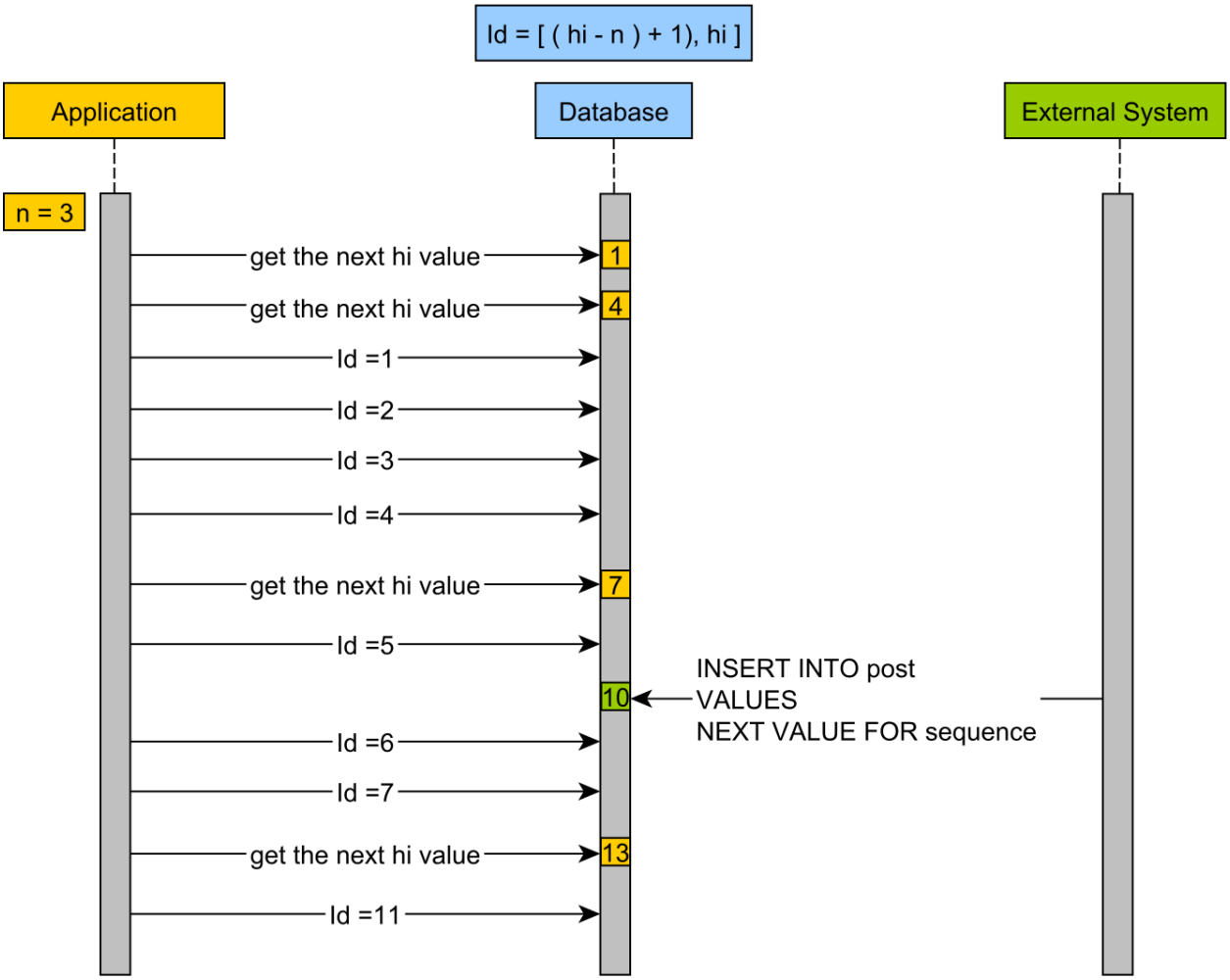

Figure 9.3: The pooled optimizer

If increment size (n) is the number of identifiers within a range, the pooled optimizer will generate identifiers with the following formula: $Id = [(hi - n) + 1, hi]$.

- The first sequence call generates the *lo* value and the second one determines the *hi* value, so the first range of identifiers is {2, 3, 4}.
- When adding the 5th entity, the pooled optimizer calls the sequence again and obtains the next *hi* value, the next identifier range being {5, 6, 7}.
- After inserting the 5th entity, an external system adds a *post* row and assigns the primary key with the value returned by the sequence call.
- The Hibernate application thread resumes and inserts the identifiers 6 and 7.
- The 8th entity requires a new sequence call, and so a new range is allocated {11, 12, 13}.

9.2.3.4.5 The pooled-lo optimizer

By changing the previous mapping to use the `pooled-lo` optimizer, the identifier generation changes as follows:

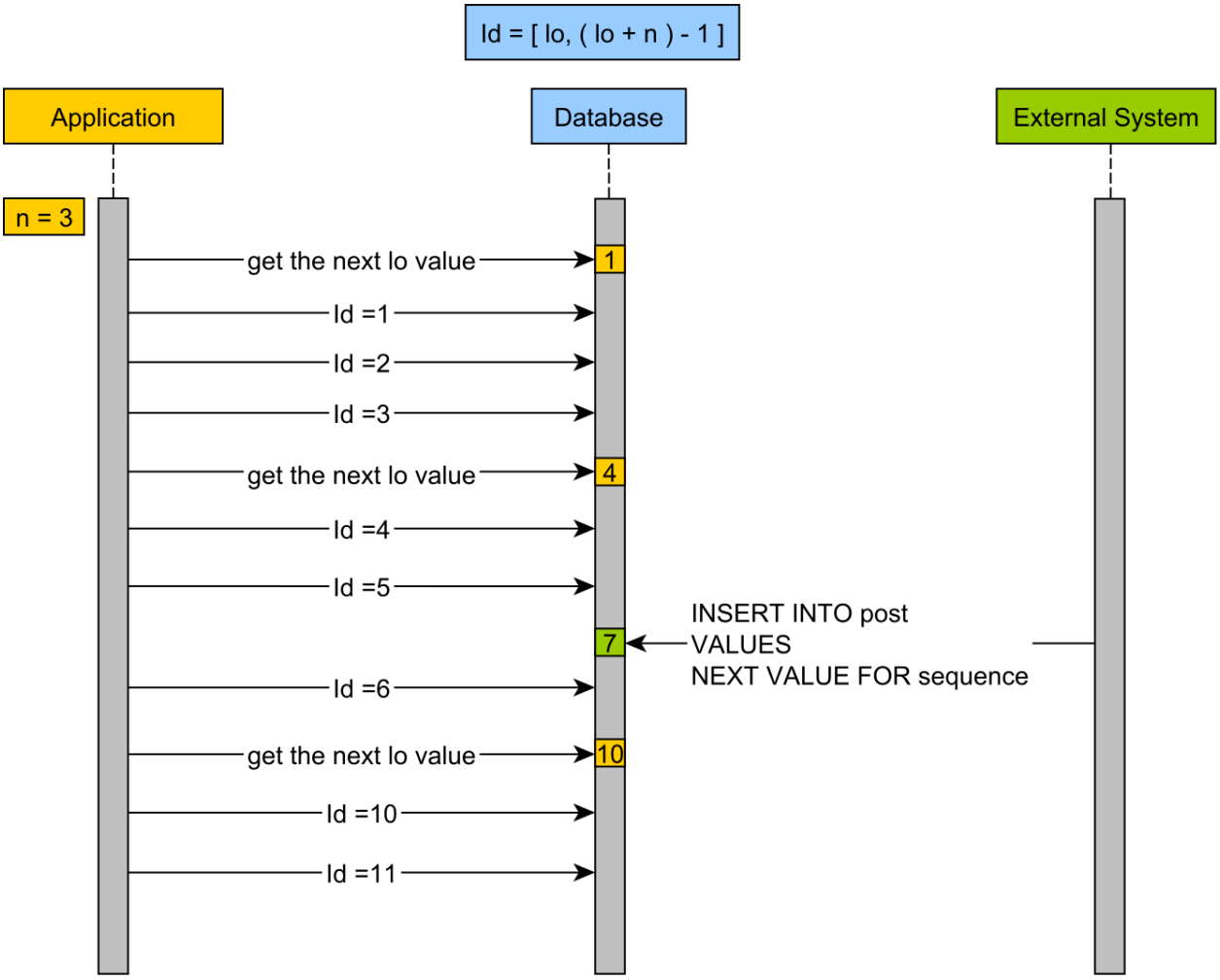

Figure 9.4: The pooled-lo optimizer

If increment size (n) is the number of identifiers within a range, the pooled-lo optimizer will generate identifiers with the following formula: $Id = [lo, (lo + n) - 1]$.

- The first sequence call generates the *lo* value, so the first range of identifiers is {1, 2, 3}.
- When adding the 4th entity, the pooled-lo optimizer calls the sequence and obtains the next *lo* value, the next identifier range being {4, 5, 6}.
- After inserting the 5th entity, an external system adds a *post* row and assigns the primary key with the value returned by the sequence call.
- The Hibernate application thread resumes and inserts the identifier 6.
- The 7th entity requires a new sequence call, and so a new range is allocated {10, 11, 12}.

9.2.3.5 Optimizer gain

To visualize the performance gain of using sequence and table generator optimizers, the following test measures the identifier allocation time when inserting 50 *post* entities and while varying the increment size (1, 5, 10, and 50).

9.2.3.5.1 Sequence generator performance gain

When using a sequence generator with the default pooled optimizer, the following 99th percentile is being recorded:

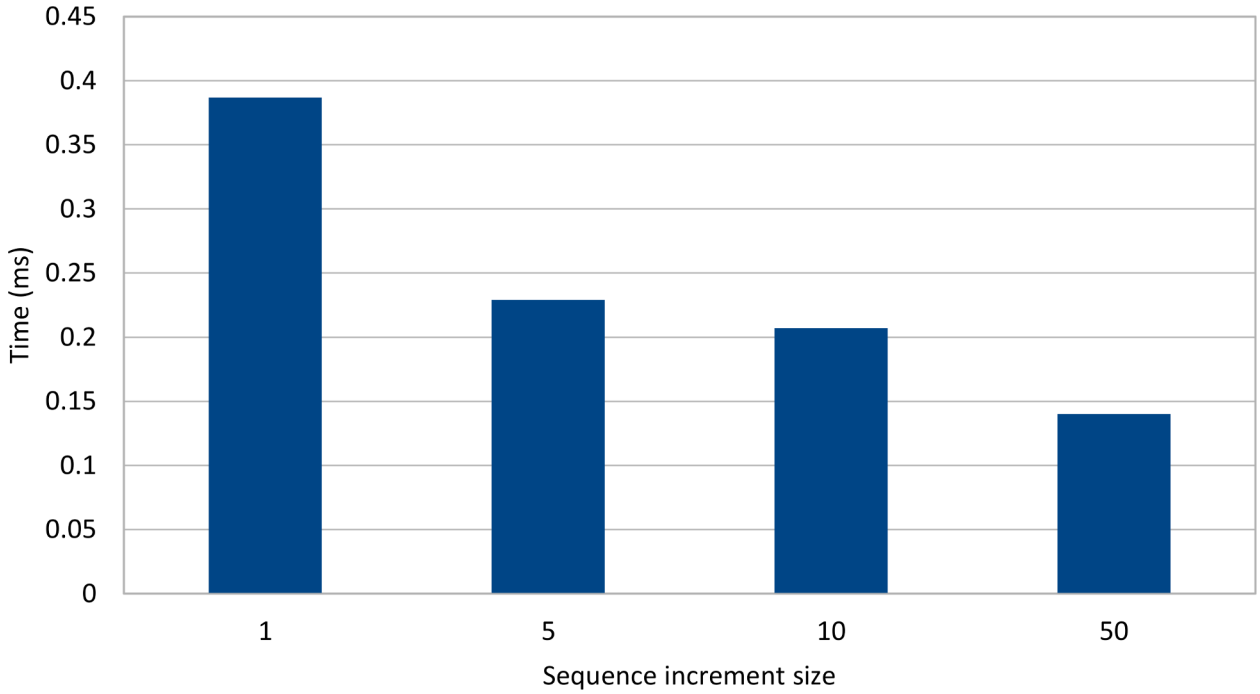

Figure 9.5: Sequence pooled optimizer gain

Database sequences are fast, but, even so, the pooled optimizer manages to reduce the identifier generation time considerably.

For write-intensive applications, the increment size needs to be adjusted according to the number of rows being inserted in one transaction.

9.2.3.5.2 Table generator performance gain

The same test suite is run against a table generator with a pooled optimizer, and the increment size also varies between 1, 5, 10, and 50. Because of the row-level locking and the extra database connection switch overhead, the table generator is less efficient than a database sequence.

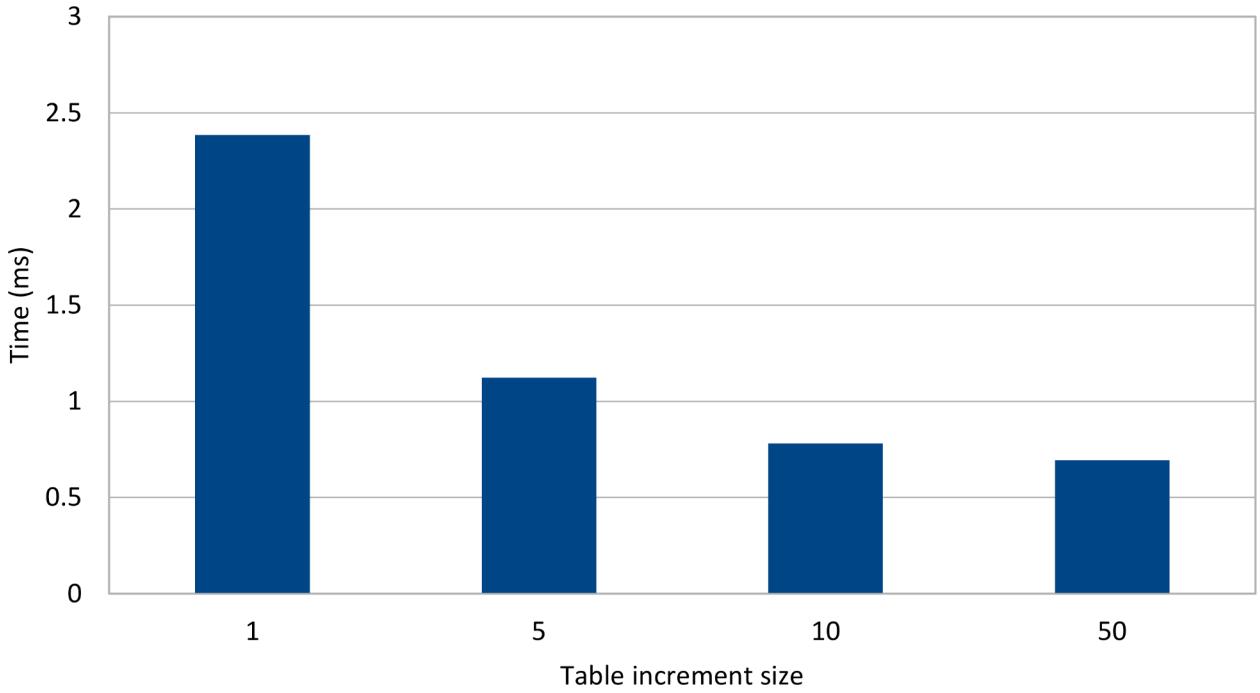

Figure 9.6: Table pooled optimizer gain

Just like with the database sequence, the pooled optimizer managed to reduce the time it took for assigning a new entity identifier.

9.2.3.6 Identifier generator performance

To evaluate the concurrency cost of each identifier generators, the following test measures the time it takes to insert 100 post entities when multiple running threads are involved. JDBC batching is enabled, and the connection pool is adjusted to accommodate the maximum number of database connection required (e.g. 32).

In reality, the application might not be configured with so many database connections, and the table generator connection acquisition cost might be even higher.

The first relational database system under test supports identity columns, so it is worth measuring how the identifier and the table generator compete. Unlike the previous test, this one measures the total time taken for inserting all entities, and not just the identifier allocation time interval.

Each test iteration increases contention by allocating more worker threads that need to execute the same database insert load.

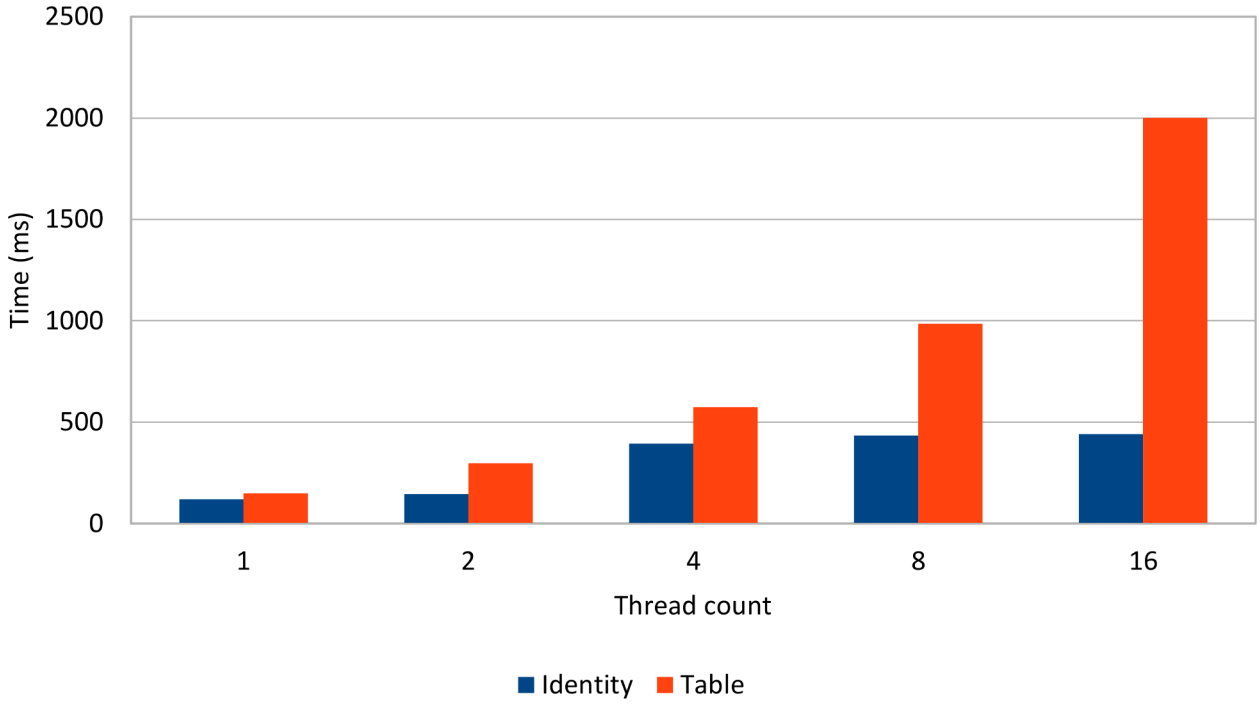

Figure 9.7: Identity vs. Table

Even if it cannot benefit from JDBC batching, the identity generator still manages to outperform the table generator, which uses a pooled optimizer with an increment size of 100.

The more threads are used, the less efficient the table generator becomes. On the other hand, identity columns scale much better with more concurrent transactions.

Even if does not support JDBC batching, native identity columns are still a valid choice, and, in future, Hibernate might even support batch inserts for those as well.

The gap between the sequence and the table generator is even higher because, just like the table generator, the sequence generator can also take advantage of the pooled optimizer as well as JDBC batch inserts.